

Scalar SQL Functions

String, Numeric, and Date/Time Functions in MySQL

Lab Manual — Lab #3

Course	Database Systems
Topic	Scalar (single-row) functions: string, numeric, and date/time
Duration	One lab session (approx. 3 hours)
Tools	MySQL 8.x, MySQL Workbench / Command Line Client
Prerequisites	Lab #1 (Filters), Lab #2 (Joins)
Next Lab	Aggregate Functions, GROUP BY, HAVING (Lab #4)

1. Learning Objectives

By the end of this lab, students will be able to:

- 1 Distinguish between **scalar functions** (one value per row) and **aggregate functions** (covered next lab).
- 2 Apply **string functions** — CONCAT, LENGTH, UPPER/LOWER, SUBSTRING, REPLACE, TRIM, LEFT/RIGHT, LPAD, LOCATE — to format and extract text.
- 3 Apply **numeric functions** — ROUND, CEIL, FLOOR, ABS, MOD, POWER, TRUNCATE — for arithmetic and rounding.
- 4 Apply **date and time functions** — NOW, CURDATE, YEAR, MONTH, DAY, DATEDIFF, DATE_ADD, DATE_FORMAT — to work with dates.
- 5 Combine multiple functions through **function nesting** (the output of one feeds another).
- 6 Combine functions with filters and joins from previous labs to clean and format real-world data.
- 7 Use the DATE_FORMAT specifier table to produce nicely formatted date output.

Outcome

Each student will write SQL queries that transform raw column data into clean, human-readable output using built-in scalar functions, producing a single .sql file with all answers ready to be pushed to their GitHub repository.

2. Session Plan (3 Hours)

Time	Activity	Mode
0:00 – 0:10	Recap of Labs #1 / #2 + lab schema setup	Setup
0:10 – 0:25	What are functions? Scalar vs aggregate; call syntax; nesting	Lecture
0:25 – 1:00	String functions with worked examples	Lecture + Demo
1:00 – 1:30	Lab Tasks Part A — String functions	Hands-on
1:30 – 1:40	Short break	—
1:40 – 2:00	Numeric functions with worked examples	Lecture + Demo
2:00 – 2:25	Date and time functions; DATE_FORMAT specifiers	Lecture + Demo
2:25 – 2:55	Lab Tasks Part B — Numeric & date functions	Hands-on
2:55 – 3:00	Wrap-up, common mistakes, deliverables	Discussion

3. The Lab Schema — Customer & Product Database

This lab uses two tables: **Customer** and **Product**. The data deliberately contains realistic messiness — extra whitespace in names, NULL values, mixed-case strings — so the string functions actually have something to do. Lab #4 will extend this schema with an OrderItem table to enable aggregation exercises.

3.1 Tables

Table	Columns	Keys
Customer	CustID, CustName, Email, City, Phone, JoinDate, DOB	PK: CustID
Product	ProdID, ProdName, Category, Price, StockQty, LaunchDate	PK: ProdID

3.2 Setup Script — Run this First

```
CREATE DATABASE IF NOT EXISTS scalar_lab;
USE scalar_lab;

DROP TABLE IF EXISTS Product, Customer;

CREATE TABLE Customer (
  CustID      INT PRIMARY KEY,
  CustName    VARCHAR(60) NOT NULL,
  Email       VARCHAR(80),
  City        VARCHAR(30),
  Phone       VARCHAR(20),
  JoinDate    DATE,
  DOB         DATE
);

CREATE TABLE Product (
  ProdID      INT PRIMARY KEY,
  ProdName    VARCHAR(60) NOT NULL,
  Category    VARCHAR(30),
  Price       DECIMAL(10,2),
  StockQty    INT,
  LaunchDate  DATE
);

-- Customers (note: extra spaces, mixed case, NULLs - on purpose)
INSERT INTO Customer VALUES
(1, ' Ali Khan ',      'ali.khan@MAIL.com',
'Lahore',      '0300-1112233', '2022-01-15', '1995-04-12'),
(2, 'Sara Iqbal',      'sara@example.com',      'Karachi',
'0301-4445566', '2022-04-22', '1998-11-20'),
(3, 'HAMZA RAZA',      'hamza@example.com',
'Lahore',      '0302-7778899', '2023-02-10', '1997-08-05'),
(4, 'Ayesha Noor',      NULL,      'Islamabad',
'0303-1234567', '2023-05-18', '1999-02-14'),
(5, 'bilal ahmed',      'bilal@MAIL.COM',      'Karachi',
'0304-2345678', '2023-09-01', '2000-06-30'),
(6, 'Fatima Sheikh', 'fatima@example.com',
NULL,      '0305-3456789', '2024-01-12', '1996-10-25'),
(7, 'Usman Tariq',      'usman@example.com', 'Lahore',      NULL,
'2024-06-30', '2001-03-18'),
(8, 'Maira Javed',      'maira@example.com', 'Islamabad',
'0307-5678901', '2024-08-25', '1994-12-09');
```

```
-- Products
INSERT INTO Product VALUES
(101,'Laptop Pro 15',      'Electronics', 185000.00, 12, '2023-03-10'),
(102,'Wireless Mouse',    'Electronics',  2500.00, 50, '2022-07-22'),
(103,'USB-C Cable',       'Electronics',   800.00, 100, '2021-11-05'),
(104,'Office Chair',      'Furniture',   18500.00,  8, '2023-01-15'),
(105,'Standing Desk',     'Furniture',   45000.50,  5, '2024-02-28'),
(106,'Notebook A4',       'Stationery',   350.00, 200, '2020-04-01'),
(107,'Ballpoint Pen 10pk','Stationery',   450.00, 150, '2020-04-01'),
(108,'Coffee Beans 1kg',  'Grocery',    1899.99, 30, '2023-09-20'),
(109,'Green Tea Box',     'Grocery',     650.00, 45, '2022-12-12'),
(110,'Bluetooth Speaker', 'Electronics',  7500.00, 18, '2024-05-18');
```

Deliberate messiness

Customer 1 (' Ali Khan ') and 6 ('Fatima Sheikh') have **extra spaces** — for TRIM practice. Customer 3 is in **uppercase**, customer 5 in **lowercase** — for UPPER/LOWER practice. Customer 4 has a **NULL email**, customer 6 has a **NULL city**, customer 7 has a **NULL phone**. Birth dates and launch dates span several years so date arithmetic produces meaningful results.

4. What Are Functions?

A SQL **function** takes zero or more inputs (arguments) and returns a single value. Functions are used inside the SELECT list, WHERE clause, ORDER BY, or anywhere a value is expected. MySQL has two broad categories:

Type	Operates on	Returns	Example
Scalar (single-row)	One row at a time	One value per row	UPPER(CustName)
Aggregate (multi-row)	A group of rows	One value per group	AVG(Price)

This lab focuses entirely on scalar functions. Aggregate functions are covered in Lab #4.

Call syntax is universal:

```
FUNCTION_NAME(argument1, argument2, ...)
```

Functions can be **nested** — the output of one function feeds another. Read nested calls inside-out.

```
UPPER(TRIM(CustName))
-- TRIM runs first, producing a clean string
-- That clean string is then passed to UPPER
```

Function names are case-insensitive

UPPER('hi'), Upper('hi'), and upper('hi') all work identically. By convention we write function names in **UPPERCASE** to distinguish them from column names.

5. String Functions

String functions transform or extract text. The most commonly used in MySQL:

Function	What it does	Example -> Result
LENGTH(s)	Number of bytes in s	LENGTH('Hello') -> 5
CHAR_LENGTH(s)	Number of characters (better than LENGTH for Unicode)	CHAR_LENGTH('Ali') -> 3
UPPER(s) / LOWER(s)	Convert case	UPPER('hi') -> 'HI'
CONCAT(a, b, ...)	Glue strings together	CONCAT('Hi ', 'Ali') -> 'Hi Ali'
CONCAT_WS(sep, a, b)	Concat with separator	CONCAT_WS('-', '2024', '01') -> '2024-01'
SUBSTRING(s, start, len)	Extract a substring (1-indexed)	SUBSTRING('Database', 1, 4) -> 'Data'
LEFT(s, n) / RIGHT(s, n)	First / last n characters	LEFT('Database', 4) -> 'Data'
TRIM(s)	Remove leading and trailing spaces	TRIM(' hi ') -> 'hi'
LTRIM(s) / RTRIM(s)	Trim left side / right side only	LTRIM(' hi ') -> 'hi'
REPLACE(s, old, new)	Replace all occurrences	REPLACE('a-b-c', '-', '/') -> 'a/b/c'
LPAD(s, len, pad) / RPAD	Pad on left / right to a length	LPAD('7', 3, '0') -> '007'

Function	What it does	Example -> Result
LOCATE(sub, s)	Position of sub in s (0 if not found)	LOCATE('a' , 'Database') -> 2
REVERSE(s)	Reverse the string	REVERSE('SQL') -> 'LQS'
REPEAT(s, n)	Repeat string n times	REPEAT('ab' , 3) -> 'ababab'

Class Example 1 — Clean up names and show in proper case:

```
SELECT CustID,
       CustName           AS Original,
       TRIM(CustName)      AS Cleaned,
       UPPER(TRIM(CustName)) AS UpperClean,
       CHAR_LENGTH(TRIM(CustName)) AS NameLength
FROM Customer;
```

Notice the **function nesting** — TRIM runs first, its output feeds UPPER and CHAR_LENGTH. Read nested functions inside-out.

Class Example 2 — Build a friendly greeting:

```
SELECT CONCAT('Hello, ', TRIM(CustName), '! Welcome back.') AS Greeting
FROM Customer;
```

Class Example 3 — Extract the email domain:

```
-- Find the position of '@', then take everything after it
SELECT CustName,
       Email,
       SUBSTRING(Email, LOCATE('@', Email) + 1) AS Domain
FROM Customer
WHERE Email IS NOT NULL;
```

Class Example 4 — Padding and slugging:

```
SELECT ProdID,
       LPAD(ProdID, 5, '0')           AS PaddedID,      -- 101 -> '00101'
       REPLACE(ProdName, ' ', '_')    AS SlugName,      -- 'Laptop Pro 15' ->
       'Laptop_Pro_15'
       LOWER(REPLACE(ProdName, ' ', '_')) AS LowerSlug
FROM Product;
```

Class Example 5 — Mask a phone number (show last 4 digits only):

```
SELECT CustName,
       Phone,
       CONCAT('XXXX-XXX', RIGHT(Phone, 4)) AS MaskedPhone
FROM Customer
WHERE Phone IS NOT NULL;
```

TRIM only removes spaces by default

TRIM('xxhixx') still returns 'xxhixx'. To trim other characters, use the full form: TRIM(BOTH 'x' FROM 'xxhixx') -> 'hi'. You can also specify LEADING or TRAILING.

6. Numeric Functions

Numeric functions perform arithmetic, rounding, and modular operations. They are commonly used to format prices, compute discounts, taxes, and partition values.

Function	What it does	Example -> Result
ROUND(n, d)	Round n to d decimal places	ROUND(3.14159, 2) -> 3.14
CEIL(n) / CEILING(n)	Smallest integer >= n	CEIL(4.2) -> 5
FLOOR(n)	Largest integer <= n	FLOOR(4.8) -> 4
TRUNCATE(n, d)	Cut off after d decimals (no rounding)	TRUNCATE(3.789, 1) -> 3.7
ABS(n)	Absolute value	ABS(-12) -> 12
MOD(a, b) or a % b	Remainder of a / b	MOD(10, 3) -> 1
POWER(a, b) / POW	a raised to the power b	POWER(2, 10) -> 1024
SQRT(n)	Square root	SQRT(81) -> 9
SIGN(n)	-1 if negative, 0 if zero, 1 if positive	SIGN(-5) -> -1
GREATEST(...) / LEAST(...)	Largest / smallest of arguments	GREATEST(5, 9, 2) -> 9
RAND()	Random number in [0, 1)	RAND() -> e.g. 0.732

Class Example 6 — Apply a 10% discount and round:

```
SELECT ProdName,
       Price           AS OriginalPrice,
       ROUND(Price * 0.90, 2) AS DiscountedPrice,
       ROUND(Price - Price*0.90, 2) AS YouSave
FROM Product;
```

Class Example 7 — CEIL vs FLOOR vs ROUND vs TRUNCATE:

```
SELECT 4.5 AS n,
       CEIL(4.5)      AS Ceil,      -- 5
       FLOOR(4.5)     AS Floor,     -- 4
       ROUND(4.5)     AS Round,     -- 5 (banker's rounding may differ)
       TRUNCATE(4.5, 0) AS Truncate; -- 4

-- For a negative number, FLOOR goes MORE negative, CEIL less negative
SELECT FLOOR(-4.2), -- -5
       CEIL(-4.2);  -- -4
```

Class Example 8 — Find even-numbered customer IDs:

```
SELECT CustID, CustName
FROM Customer
WHERE MOD(CustID, 2) = 0;
-- Equivalent: WHERE CustID % 2 = 0
```

Class Example 9 — Compute 17% sales tax on each product:

```
SELECT ProdName,
       Price,
       ROUND(Price * 0.17, 2) AS Tax,
       ROUND(Price * 1.17, 2) AS PriceWithTax
FROM Product
ORDER BY PriceWithTax DESC;
```

7. Date and Time Functions

Date functions are essential for reporting — almost every business question involves a time window (this month, last quarter, customers older than 1 year, etc.). MySQL stores dates in 'YYYY-MM-DD' format and datetimes in 'YYYY-MM-DD HH:MM:SS'.

Function	What it does	Example -> Result
NOW()	Current date AND time	2026-05-17 14:32:10
CURDATE() / CURRENT_DATE	Today's date (no time)	2026-05-17
CURTIME()	Current time only	14:32:10
YEAR(d) / MONTH(d) / DAY(d)	Extract one component	YEAR('2024-08-25') -> 2024
DAYNAME(d) / MONTHNAME(d)	Name of day / month	DAYNAME('2024-01-01') -> 'Monday'
DAYOFWEEK(d)	1=Sunday ... 7=Saturday	DAYOFWEEK('2024-01-01') -> 2
DAYOFYEAR(d)	Day of the year (1–366)	DAYOFYEAR('2024-02-01') -> 32
DATEDIFF(end, start)	Difference in DAYS (end - start)	DATEDIFF('2024-12-31', '2024-01-01') -> 365
DATE_ADD(d, INTERVAL n unit)	Add n units (DAY, MONTH, YEAR, ...)	DATE_ADD('2024-01-01', INTERVAL 7 DAY)
DATE_SUB(d, INTERVAL n unit)	Subtract n units	DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
TIMESTAMPDIFF(unit, a, b)	Diff in chosen unit (YEAR, MONTH, DAY, ...)	TIMESTAMPDIFF(YEAR, '2000-01-01', CUR DATE())
DATE_FORMAT(d, fmt)	Format a date as text	See specifiers below
STR_TO_DATE(s, fmt)	Parse text into a date	STR_TO_DATE('25-08-2024', '%d-%m-%Y')

7.1 DATE_FORMAT Specifiers (most useful)

Specifier	Meaning	Example
%Y	4-digit year	2024
%y	2-digit year	24
%m	Month number (01–12)	08
%M	Month name	August
%b	Abbreviated month	Aug
%d	Day of month (01–31)	25
%D	Day with suffix	25th
%W	Weekday name	Sunday
%a	Abbreviated weekday	Sun
%H	Hour (00–23)	14

Specifier	Meaning	Example
%i	Minutes (00–59)	30
%s	Seconds (00–59)	45

Class Example 10 — Extract year, month, day of week from launch dates:

```
SELECT ProdID,
       ProdName,
       LaunchDate,
       YEAR(LaunchDate)    AS LaunchYear,
       MONTH(LaunchDate)   AS LaunchMonth,
       DAYNAME(LaunchDate) AS LaunchDay
FROM Product;
```

Class Example 11 — How long has each customer been with us?

```
SELECT CustName,
       JoinDate,
       DATEDIFF(CURDATE(), JoinDate)          AS DaysAsMember,
       TIMESTAMPDIFF(YEAR, JoinDate, CURDATE()) AS YearsAsMember
FROM Customer;
```

Class Example 12 — Compute each customer's current age:

```
SELECT CustName,
       DOB,
       TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age
FROM Customer;
-- TIMESTAMPDIFF with YEAR correctly handles whether the birthday has passed this year
```

Class Example 13 — Pretty-print a date:

```
SELECT CustName,
       JoinDate,
       DATE_FORMAT(JoinDate, '%W, %D %M %Y') AS NiceDate
FROM Customer;
-- '2022-01-15' -> 'Saturday, 15th January 2022'
```

Class Example 14 — Customers who joined in the last 12 months:

```
SELECT CustName, JoinDate
FROM Customer
WHERE JoinDate >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH);
```

Class Example 15 — Products launched in Q1 of any year:

```
SELECT ProdName, LaunchDate
FROM Product
WHERE MONTH(LaunchDate) IN (1, 2, 3)
ORDER BY LaunchDate;
```

DATEDIFF vs TIMESTAMPDIFF

DATEDIFF always returns days. TIMESTAMPDIFF lets you pick the unit (YEAR, MONTH, DAY, HOUR, MINUTE, SECOND) and correctly handles month length and leap years — essential for computing ages and tenure.

8. Lab Tasks

Solve each task in MySQL using the schema from section 3. Save all queries in a single file named `RollNo_ScalarFunctions.sql`, with a comment line before each answer (e.g. `-- Task A1`).

Part A — String Functions

#	Task
A1	List each customer's name with leading and trailing spaces removed, alongside the original. Show CustID, original CustName, CleanedName.
A2	Display every customer name in UPPERCASE and the same name in lowercase. Show CustID, UpperName, LowerName.
A3	For each customer, show their trimmed name and the number of characters in it.
A4	Build a greeting column for each customer: 'Dear <trimmed name>, welcome!'
A5	For customers who have an email, extract the part before the '@' (the username). Show CustName and Username.
A6	For customers who have an email, extract the domain (everything after '@'). Show CustName and Domain.
A7	Show each customer's name with the first 3 characters only. (Hint: LEFT.)
A8	Mask each phone number: show the country/area code (first 4 chars), then 'XXX-XXXX'. Skip customers with no phone.
A9	Show each product name with all spaces replaced by hyphens. Show ProdID and SlugName.
A10	Show each ProdID padded to 5 digits with leading zeros (e.g. 101 -> '00101').
A11	Show product names that contain the word 'Pro' anywhere (use LIKE OR LOCATE) along with the position of 'Pro'.
A12	Display every customer's first name only (the part before the first space in the trimmed name). (Hint: SUBSTRING with LOCATE.)

Part B — Numeric and Date/Time Functions

#	Task
B1	Apply a 15% discount to every product. Show ProdName, Price, DiscountedPrice rounded to 2 decimals.
B2	For each product, compute 17% sales tax and the final price (price + tax). Show ProdName, Tax, PriceWithTax.
B3	Compute the floor and ceiling of every product's price divided by 1000. Show ProdName, Price, FloorVal, CeilVal.
B4	Round each product's price to the nearest hundred. (Hint: ROUND with negative second argument, e.g. ROUND(Price, -2).)
B5	List products with an odd ProdID. (Hint: MOD.)
B6	For each customer, show the year, month name, and day of the week they joined.
B7	Display each customer's date of birth formatted as 'DD-Month-YYYY' (e.g. '12-April-1995').
B8	Compute each customer's current age in years. Show CustName, DOB, Age. (Use TIMESTAMPDIFF.)
B9	Compute how many days ago each customer joined (from today). Show CustName, JoinDate, DaysSinceJoin.
B10	List customers who joined in the year 2023 . Use a date function, not BETWEEN.
B11	List products launched in any year's Q4 (October, November, December).
B12	Find customers who joined within the last 6 months from today. Use DATE_SUB.
B13	Calculate each product's 'age' in days (days since launch). Show ProdName, LaunchDate, AgeInDays.
B14	Compute the date exactly 90 days from each product's LaunchDate. Show ProdName, LaunchDate, NinetyDaysLater.
B15	Combined challenge: produce a single column called 'Summary' formatted as 'Hello ALI KHAN, age 31, joined Jan 2022' for every customer. (Hint: CONCAT + UPPER + TRIM + TIMESTAMPDIFF + DATE_FORMAT.)

Tip

When nesting functions, write the innermost call first, run it, verify the result, then wrap it in the next function. Building a 4-function chain in one go almost guarantees a syntax error you cannot locate.

9. Assessment Problem (Solve Individually)

Scenario — Employee Database. A small company stores employee records in a single table. Build the schema in MySQL, load the provided sample data, and answer the ten questions that follow. *This problem is graded.*

9.1 Setup Script

```
CREATE DATABASE IF NOT EXISTS emp_lab;
USE emp_lab;

DROP TABLE IF EXISTS Employee;

CREATE TABLE Employee (
    EmpID      INT PRIMARY KEY,
    FullName   VARCHAR(60) NOT NULL,
    Email      VARCHAR(80),
    Phone      VARCHAR(20),
    DOB        DATE,
    HireDate   DATE,
    Salary     DECIMAL(10,2),
    City       VARCHAR(30),
    JobTitle    VARCHAR(40)
);

INSERT INTO Employee VALUES
(2001, ' ahmad raza',      'ahmad@firm.com', '0300-1112233', '1990-04-12', '2018-09-01',
120000.50, 'Lahore', 'Senior Engineer'),
(2002, 'Sara Imran',      'SARA@FIRM.COM',   '0301-4445566', '1992-11-20', '2019-03-15',
95000.00, 'Karachi', 'Software Engineer'),
(2003, 'BILAL KHAN',      'bilal@firm.com', '0302-7778899', '1993-08-05', '2020-01-20',
85000.75, 'Lahore', 'QA Engineer'),
(2004, 'Fatima Ali',      NULL,              '0303-1234567', '1991-02-14', '2017-11-10',
110000.00, 'Islamabad', 'Manager'),
(2005, 'Hira Yousaf',      'hira@firm.com',   NULL,              '1995-06-30', '2021-04-05',
70000.00, NULL, 'Accountant'),
(2006, 'Zain Abbas ',      'zain@firm.com',   '0305-3456789', '1994-10-25', '2022-08-30',
78000.40, 'Karachi', 'Designer'),
(2007, 'Mehwish Anwar',    'mehwish@FIRM.com', '0306-4567890', '1989-12-09', '2016-07-22',
125000.00, 'Lahore', 'Director'),
(2008, 'Talha Hussain',    'talha@firm.com', '0307-5678901', '1996-03-18', '2023-01-09',
60000.00, 'Islamabad', 'HR Officer'),
(2009, 'Areeba Yasin',     'areeba@firm.com', '0308-6789012', '1990-07-22', '2019-09-12',
90000.99, 'Lahore', 'Analyst'),
(2010, 'Hassan Ahmed',     'hassan@firm.com', '0309-7890123', '1997-01-30', '2024-02-18',
65000.00, 'Karachi', 'Junior Developer');
```

9.2 Questions

#	Question	Marks
Q1	Display each employee's full name with leading/trailing spaces removed and in proper UPPERCASE. Show EmpID, original FullName, CleanedName.	8
Q2	For employees with a recorded email, extract the username (part before '@'). Show FullName and Username.	8
Q3	Mask each phone number: show the first 4 characters then 'XXX-XXXX'. Skip employees with no phone.	8

#	Question	Marks
Q4	Generate a corporate email for each employee: lowercase the trimmed FullName, replace spaces with dots, then append '@company.com'. (e.g. 'AHMAD RAZA' -> 'ahmad.raza@company.com'.) Show FullName and GeneratedEmail.	12
Q5	Apply a 12.5% pay raise to every employee. Show FullName, Salary, NewSalary rounded to 2 decimals.	8
Q6	Round every salary down to the nearest thousand. Show FullName, Salary, RoundedSalary.	8
Q7	Compute each employee's current age and years of service. Show FullName, AgeYears, YearsOfService. (Use TIMESTAMPDIFF.)	12
Q8	Format every employee's HireDate as 'DD-Mon-YYYY' (e.g. '01-Sep-2018'). Show FullName and FormattedHireDate.	8
Q9	List employees who were hired in the year 2019 or later. Use a date function rather than BETWEEN.	8
Q10	Combined challenge: produce a single column 'Profile' formatted as: 'AHMAD RAZA Lahore Senior Engineer Joined: 01-Sep-2018 Age: 35'. Use CONCAT, UPPER, TRIM, DATE_FORMAT, TIMESTAMPDIFF.	20

9.3 Submission

- Push your RollNo_ScalarFunctions.sql file to your existing GitHub repository in a folder named Lab3-ScalarFunctions/.
- The file must contain: the setup script (or a comment confirming you ran it), all Lab Task answers (Parts A and B), and all 10 Assessment answers, each labelled with a comment (e.g. -- Q5).
- Share the GitHub link to that folder before the deadline announced in class.

10. Common Mistakes to Avoid

- Forgetting that TRIM only removes **spaces** by default. Use TRIM(BOTH 'x' FROM s) to remove other characters.
- Trying to compare to NULL using a function: WHERE LENGTH(City) = 0 still excludes NULL rows. NULL passed to a function usually returns NULL.
- Using LENGTH instead of CHAR_LENGTH for multi-byte text — LENGTH returns byte count, not character count.
- Forgetting that SUBSTRING is **1-indexed**, not 0-indexed.
- Confusing LOCATE(sub, str) argument order — substring comes *first*.
- Calling DATEDIFF(start, end) the wrong way around — the formula is **end minus start**.
- Subtracting dates with - instead of DATEDIFF — works for some types but not safely for DATE columns.
- Using DATEDIFF / 365 to get age in years — it's approximate and may be off by one near birthdays. Use TIMESTAMPDIFF(YEAR, ...) instead.
- Forgetting the INTERVAL keyword in DATE_ADD/DATE_SUB: DATE_ADD(d, 7 DAY) is wrong; DATE_ADD(d, INTERVAL 7 DAY) is right.
- Mixing up %m (number, 01–12) and %M (full month name) in DATE_FORMAT. Case matters.

11. Appendix — Quick Reference

```
-- STRING
LENGTH(s)          CHAR_LENGTH(s)          UPPER(s)          LOWER(s)
CONCAT(a, b)        CONCAT_WS(sep, ...)
SUBSTRING(s, start, len)  LEFT(s, n)      RIGHT(s, n)
TRIM(s)             LTRIM(s)             RTRIM(s)          REPLACE(s, old, new)
LPAD(s, n, p)        RPAD(s, n, p)          LOCATE(sub, s)
REVERSE(s)          REPEAT(s, n)

-- NUMERIC
ROUND(n, d)          CEIL(n)              FLOOR(n)           TRUNCATE(n, d)
ABS(n)               MOD(a, b)            POWER(a, b)         SQRT(n)
SIGN(n)              GREATEST(a, b, ...) LEAST(a, b, ...)

-- DATE/TIME
NOW()                CURDATE()            CURTIME()
YEAR(d)              MONTH(d)             DAY(d)              DAYNAME(d)
DAYOFWEEK(d)         DAYOFYEAR(d)         MONTHNAME(d)

DATEDIFF(end, start)                                -- result in DAYS
TIMESTAMPDIFF(YEAR, start, end)                     -- result in chosen unit
DATE_ADD(d, INTERVAL n DAY/MONTH/YEAR)
DATE_SUB(d, INTERVAL n DAY/MONTH/YEAR)

-- DATE_FORMAT specifiers
DATE_FORMAT(d, '%Y-%m-%d')    -- 2024-08-25
DATE_FORMAT(d, '%d %M %Y')    -- 25 August 2024
DATE_FORMAT(d, '%W, %D %b %y') -- Sunday, 25th Aug 24

-- NESTING (read inside-out)
UPPER(TRIM(CustName))        -- trim first, then uppercase
```

```
SUBSTRING(Email, LOCATE('@', Email) + 1)      -- everything after '@'
```

End of Lab Manual #3 — Scalar SQL Functions. Next lab: Aggregate Functions.